СЕССИОННОЕ ЗАДАНИЕ
ПО ДИСЦИПЛИНЕ «ОСНОВЫ КРИПТОГРАФИИ»

Выполнил(а): Стрижак Тимофей Андреевич

Группа: М26 - 545

1. Трехсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Для распределения сеансовых ключей между двумя участниками может применяться протокол Нидхэма — Шрёдера с симметричным шифрованием. В нем участвуют три стороны: А и Б — абоненты, которым необходимо установить защищенное соединение, и S — доверенный сервер распределения ключей (Key Distribution Center, KDC).

До начала обмена у каждой пары «абонент — сервер» имеется собственный долговременный секретный ключ: КАС известен только А и серверу S, а КБS — только Б и серверу S. Эти ключи используются для безопасной передачи нового сеансового ключа. Обозначим сеансовый ключ для связи А и Б через КАБ, а случайное одноразовое число (nonce) — через NA или NБ.

Обмен выполняется по следующим этапам:

1) А → S: идентификаторы участников IDA и IDБ, а также случайное число NA. А запрашивает у сервера сеансовый ключ для связи с Б. Случайное число позволяет А проверить, что ответ сервера относится именно к текущему запросу.

2) S → А: сервер генерирует новый случайный сеансовый ключ КАБ и формирует два элемента. Первый — {КАБ, IDБ, NA}КАС, зашифрованный долговременным ключом КАС. Второй — «билет» для Б: {КАБ, IDA}КБS, зашифрованный ключом КБS. Сервер отправляет оба элемента участнику А. А

расшифровывает первую часть, проверяет NA и получает КАБ. Билет для Б А расшифровать не может и не должен; он пересылает его без изменений.

3) А → Б: А отправляет Б билет {КАБ, IDA}КБS. Б расшифровывает его своим долговременным ключом КБS и получает сеансовый ключ КАБ, а также идентификатор А. Таким образом, Б получает ключ, предназначенный для связи именно с А.

4) Б → А: для проверки того, что А действительно владеет КАБ, Б отправляет зашифрованное этим ключом случайное число NБ: {NБ}КАБ.

5) А → Б: А расшифровывает сообщение, выполняет предусмотренное протоколом преобразование над NБ (например, уменьшает число на единицу) и отправляет результат под защитой КАБ: {f(NБ)}КАБ. Б проверяет ответ. Успешная проверка подтверждает, что А также владеет сеансовым ключом.

Итог: участники А и Б получают общий сеансовый симметричный ключ КАБ, известный им и доверенному серверу S. Ключ используется для защиты последующего обмена данными между А и Б. Долговременные ключи КАС и КБS служат для безопасной доставки сеансового ключа, а не для шифрования всего пользовательского обмена.

2. Формальные модели шифров

Формальная модель криптосистемы описывает множества допустимых сообщений, криптограмм и ключей, а также алгоритмы шифрования и расшифрования. В базовой модели симметричной криптосистемы задается пятерка (M, C, K, E, D), где M — множество открытых сообщений; C — множество шифртекстов (криптограмм); K — множество ключей; E — семейство функций шифрования E_k; D — семейство функций расшифрования D_k.

Для каждого ключа k алгоритм шифрования преобразует сообщение $m \in M$ в криптограмму $c \in C$: $c = E_k(m)$. Алгоритм расшифрования восстанавливает

исходное сообщение: $m = D_k(c)$. Основное условие корректности имеет вид $D_k(E_k(m)) = m$ для любого допустимого сообщения m и ключа k. В симметричной системе один и тот же секретный ключ (или общий секретный материал) используется обеими сторонами для шифрования и расшифрования.

В асимметричной криптосистеме каждому участнику соответствует пара ключей: открытый pk и закрытый sk. Открытый ключ может распространяться свободно, а закрытый хранится в тайне. Шифрование выполняется открытым ключом получателя: c = E_pk(m), а расшифрование — соответствующим закрытым ключом: m = D_sk(c). Условие корректности записывается как D_sk(E_pk(m)) = m. Такая модель позволяет передавать конфиденциальные данные без предварительной передачи общего секретного ключа по защищенному каналу.

В практических криптосистемах алгоритмы могут быть вероятностными: шифрование использует случайное значение (например, вектор инициализации или случайный параметр), поэтому одно и то же сообщение при одном ключе может давать разные криптограммы. В таком случае случайный параметр является частью входа алгоритма, а корректность расшифрования должна сохраняться для всех допустимых значений этого параметра.

3. Принципы построения поточных систем шифрования

Поточный шифр преобразует открытый текст последовательно, обычно побитно или поблочно небольшими фрагментами. На основе секретного ключа и, как правило, вектора инициализации (IV) формируется псевдослучайная последовательность — гамма. Открытый текст объединяется с гаммой, чаще всего операцией XOR: $C_i = P_i \oplus \gamma_i$. При расшифровании выполняется та же операция: $P_i = C_i \oplus \gamma_i$, поскольку XOR является обратной самому себе.

Основные принципы построения поточных шифров следующие:

1) Криптографическая непредсказуемость гаммы. Последовательность гаммы должна быть вычислительно неотличима от случайной для противника, не знающего ключ. Это затрудняет предсказание следующих элементов гаммы и восстановление открытого текста.

2) Большой период и отсутствие коротких циклов. Генератор гаммы должен выдавать длинную последовательность до повторения. Это снижает риск повторного использования одних и тех же фрагментов гаммы и связанных с этим утечек информации.

3) Зависимость гаммы от секретного ключа и уникального IV. Ключ обеспечивает секретность, а IV задает начальное состояние генератора. Для одного ключа IV должен использоваться согласно требованиям конкретного алгоритма; в частности, его повторение там, где это запрещено, может привести к повторению гаммы и раскрытию связей между сообщениями.

4) Равномерные статистические свойства. Биты гаммы должны быть сбалансированы, а последовательность не должна иметь выявляемых статистических закономерностей. Это препятствует статистическому анализу шифртекста.

5) Высокая скорость и экономичность реализации. Генерация гаммы и преобразование данных должны выполняться быстро и с небольшими затратами памяти и вычислительных ресурсов. Это важно для потоковой передачи, сетевых соединений и устройств с ограниченными ресурсами.

6) Синхронизация отправителя и получателя. Обе стороны должны генерировать одинаковую гамму для соответствующих позиций сообщения. Потеря или вставка фрагментов данных в синхронных потоковых схемах может нарушить соответствие гаммы и текста, поэтому протокол должен обеспечивать восстановление или поддержание синхронизации.

7) Контроль ошибок и целостности. Само поточное шифрование обычно не гарантирует обнаружение преднамеренного изменения шифртекста. Поэтому

для защиты от модификации данных следует применять механизм аутентификации, например код аутентификации сообщения (MAC) или режим аутентифицированного шифрования.

4. Пример режима, в котором биективность шифрующих отображений базового режима не требуется

Примером является режим гаммирования с обратной связью по шифртексту (CFB, Cipher Feedback). В этом режиме блочный алгоритм используется для выработки гаммы, а не для непосредственного обратимого преобразования каждого блока открытого текста. Поэтому для однозначного расшифрования не требуется применять обратное отображение базового шифра.

Пусть E_k — преобразование, выполняемое базовым алгоритмом под ключом k, а R_i — содержимое регистра обратной связи. Для простоты рассмотрим полный блок: сначала вычисляется $E_k(R_i)$, затем результат складывается по XOR с блоком открытого текста P_i, образуя шифртекст C_i: $C_i = P_i \oplus E_k(R_i)$. После этого шифртекст C_i помещается в регистр обратной связи для формирования следующего состояния.

При расшифровании получатель, имея тот же ключ и начальное состояние, вычисляет ту же величину $E_k(R_i)$ и получает $P_i = C_i \oplus E_k(R_i)$. Таким образом, расшифрование использует прямое преобразование E_k, а не обратное D_k. Даже если отображение E_k само по себе не является биективным, восстановление P_i остается однозначным: XOR с известной гаммой является взаимно однозначным преобразованием.

Следовательно, в режиме CFB биективность базового шифрующего отображения не является необходимым условием однозначного расшифрования. Существенно, чтобы обе стороны одинаково вычисляли последовательность обратной связи и гамму, а также использовали одинаковые ключ и начальное состояние.

5. Распространение ошибки в режиме CFB при размере сегмента 8 бит

В условии используется режим CFB с размером блока базового блочного шифра 64 бита и размером сегмента обратной связи 8 бит. Следовательно, каждый 64-битовый блок состоит из восьми 8-битовых сегментов (байтов). При расшифровании очередной сегмент открытого текста получается как XOR принятого сегмента шифртекста с соответствующими 8 битами результата шифрующего преобразования регистра обратной связи.

Пусть один бит сегмента шифртекста C_i был искажен. В текущем сегменте открытого текста P_i изменится ровно соответствующий бит, поскольку гамма для этого шага уже сформирована из предыдущего состояния регистра, а операция XOR непосредственно связывает биты C_i и P_i.

Однако искаженный 8-битовый сегмент шифртекста затем попадает в регистр обратной связи. В CFB-8 регистр сдвигается на 8 бит после каждого шага. Поэтому в течение следующих восьми шагов ошибочный сегмент остается частью регистра и влияет на вход базового шифра. Из-за этого следующие восемь 8-битовых сегментов открытого текста также будут, вообще говоря, искажены непредсказуемым образом. После восьми сдвигов ошибочный сегмент полностью вытесняется из 64-битового регистра, и дальнейшая расшифровка снова синхронизируется.

Итак, ошибка затрагивает текущий сегмент и восемь следующих сегментов — всего девять последовательных байтов. В текущем байте гарантированно инвертируется один соответствующий бит; в следующих восьми байтах значения в общем случае искажаются из-за изменения входа шифрующего преобразования.

Поскольку один 64-битовый блок содержит восемь байтов, любые девять последовательных байтов затрагивают два 64-битовых блока: часть первого блока и часть следующего. Следовательно, при заданных размерах искажаются 2 блока открытого текста. Это не означает, что все 128 бит этих двух блоков

обязательно изменятся: в первом затронутом сегменте гарантированно меняется один бит, а в следующих восьми сегментах возможны многочисленные изменения битов.

6. Размер хэш-значения для вероятности коллизии не более 10^{-9}

Хэш-функция принимает значения из множества двоичных n-мерных векторов. Число возможных хэш-значений равно 2^n. Для идеальной хэш-функции при равномерном распределении результатов вероятность того, что два независимых случайных сообщения получат одинаковое хэш-значение, равна обратной величине числа возможных результатов:

P(коллизия для одной случайной пары) = $1 / 2^n$.

По условию эта вероятность не должна превышать 10^{-9}. Запишем требуемое неравенство:

$1 / 2^n \leq 10^{-9}$.

Преобразуем его:

$2^n \geq 10^9$.

Применим двоичный логарифм к обеим частям:

$n \geq \log_2(10^9) = 9 \cdot \log_2(10)$.

Так как $\log_2(10) \approx 3{,}321928$, получаем:

$n \geq 9 \cdot 3{,}321928 \approx 29{,}897352$.

Поскольку n — целое число, минимальное допустимое значение округляется вверх: n = 30.

Проверка: при n = 30 вероятность коллизии для одной случайной пары равна $1 / 2^{30} \approx 9{,}313 \cdot 10^{-10}$, что меньше 10^{-9}. При n = 29 вероятность равна $1 / 2^{29} \approx 1{,}863 \cdot 10^{-9}$, что больше заданного ограничения.

Ответ: минимальная длина хэш-значения составляет 30 бит. Здесь рассматривается вероятность совпадения результатов для одной случайной пары сообщений. Парадокс дней рождений применяется к другой постановке —

вероятности появления хотя бы одной коллизии среди множества из нескольких сообщений; для одной заранее рассматриваемой пары точная вероятность равна 2^{-n}.